

Assignment 1 – Timothy Hansen

Repo link: <https://github.com/TimothyH3/ser321-spring26B-tjhanse9.git>

Part 1: Linux, Setup

Command line tasks

1. Create a directory named "cli_assignment".
`mkdir cli_assignment`
2. Change the current working directory to the new directory created in the previous step.
`cd cli_assignment`
3. Create a new file named "stuff.txt". Use the touch command to do this. Read about the touch command using the manual (man) pages.
`touch stuff.txt`
4. Add some text (multiple lines) to this text file using the cat command.
`cat > stuff.txt`
5. Count the number of words and the number of lines in the file "stuff.txt".
`wc stuff.txt`
6. Append more text to the file "stuff.txt".
`cat >> stuff.txt`
7. In the current working directory, create a new directory "draft".
`mkdir draft`
8. Move the "stuff.txt" file to the directory "draft".
`mv stuff.txt draft/`
9. Change your working directory to "draft" and create a hidden file named "secret.txt".
`cd draft && touch .secret.txt`
10. Create a new directory ("final") as a copy of the "draft" directory (final should be on the same level as draft) using the copy command.
`cd .. && cp -r draft final`
11. Rename the "draft" directory to "draft.remove". Use the mv command for this.
`mv draft draft.remove`
12. Move the "draft.remove" directory inside the "final" directory. Use the mv command for this.
`mv draft.remove final/`

13. From inside the "cli_assignment" directory, list all the files and sub-directories and their permissions.

```
ls -l
```

14. List the contents of the given file "NASA_access_log_Aug95.gz" without extracting it. (The file should be on the same level as your "cli_assignment" directory)

```
zcat ../NASA_access_log_Aug95.gz | head
```

15. Extract the given file "NASA_access_log_Aug95.gz".

```
gunzip -k ../NASA_access_log_Aug95.gz
```

16. Rename the extracted file to "logs.txt".

```
mv ../NASA_access_log_Aug95 logs.txt
```

17. Move the file "logs.txt" to the "cli_assignment" directory.

```
mv logs.txt cli_assignment/
```

18. Read the top 100 lines of the file "logs.txt".

```
head -n 100 logs.txt
```

19. Create a new file "logs_top_100.txt" containing the top 100 lines using I/O redirection.

```
head -n 100 logs.txt > logs_top_100.txt
```

20. Read the bottom 100 lines of the file "logs.txt".

```
tail -n 100 logs.txt
```

21. Create a new file "logs_bottom_100.txt" containing the bottom 100 lines using I/O redirection.

```
tail -n 100 logs.txt > logs_bottom_100.txt
```

22. Create a new file "logs_snapshot.txt" by concatenating files "logs_top_100.txt" and "logs_bottom_100.txt".

```
cat logs_top_100.txt logs_bottom_100.txt > logs_snapshot.txt
```

23. Now append (to the end) to the "logs_snapshot.txt" the line "tjhanse9: This is a great assignment" and the current date.

```
echo "tjhanse9: This is a great assignment 3/16/26" >> logs_snapshot.txt
```

24. Read the file "logs.txt" using the less command.

```
less logs.txt
```

25. Using the given file "marks.csv" (delimited by %), print the column "student_names" without the header (you can use the column num as index). Use the cut command and I/O redirection. (This file should be on the same level as your "cli_assignment" directory).

```
cut -d'%' -f1 ../marks.csv | tail -n +2
```

26. Using the given file "marks.csv", print the sorted list of marks in "subject_3". Use the sort command piped with the cut command.

```
tail -n +2 ../marks.csv | cut -d'%' -f4 | sort -n
```

27. Using the given file "marks.csv", print the average marks for "subject_2" (it is ok to use awk).

```
awk -F',' 'NR>1 {sum+=$3; count++} END {print sum/count}' ../marks.csv
```

28. Save the average into a new file "done.txt" inside of the "cli_assignment" directory.

```
awk -F',' 'NR>1 {sum+=$3; count++} END {print sum/count}' ../marks.csv > done.txt
```

29. Move "done.txt" into your "final" directory.

```
mv done.txt final/
```

30. Rename the "done.txt" file to "average.txt".

```
mv final/done.txt final/average.txt
```

2.2: Running examples

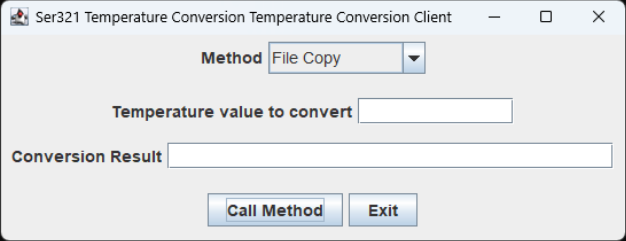
FileCopy Program

```
You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

See https://docs.gradle.org/7.6/userguide/command_line_interface.html#sec:command_line_warnings

BUILD SUCCESSFUL in 6m 33s
2 actionable tasks: 2 executed
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Threads\FileCopy> gradle run
Starting a Gradle Daemon (subsequent builds will be faster)

> Task :run
beginning the file copy
You selected the file: tmp.jar
Opening the file: tmp.jar
Writing the file: tmp.jar
Completed the file copy.
beginning the file copy
You selected the file: paper-1.21.11-99.jar
Opening the file: paper-1.21.11-99.jar
Writing the file: tmp.jar
written 4096 of 54822933 bytes
written 8192 of 54822933 bytes
written 12288 of 54822933 bytes
written 16384 of 54822933 bytes
written 20480 of 54822933 bytes
written 24576 of 54822933 bytes
written 28672 of 54822933 bytes
<=====----> 75% EXECUTING [57s]
> :run
```



Description: This is a temperature conversion client that can simultaneously run a simulated file copy in the background via multithreading. It uses a separate background thread for the file copy so the program keeps the main window remains fully responsive and doesn't freeze while in progress.

JSON Program

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Network\JSON> gradle run

> Task :run
ASU
Poly
[{"firstName":"John","lastName":"Doe"}, {"firstName":"Anna","lastName":"Smith"}, {"firstName":"Peter","lastName":"Jones"}]
John
Anna
Peter

Deprecated Gradle features were used in this build, making it incompatible with Gradle 8.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

See https://docs.gradle.org/7.6/userguide/command_line_interface.html#sec:command_line_warnings

BUILD SUCCESSFUL in 1s
2 actionable tasks: 2 executed
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Network\JSON> |

```

Description: This program parses and makes changes to JSON data by extracting certain parameters from a given string. It filters only the first names from a list of employees, saves them into names.json, and prints it to the console.

GroupJson Program

```

PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Serialization\GroupJson> gradle run

> Task :run
Group: Administration has users: Tim, Joe, Sue,
Administration group as json string: {"Joe":{"userPwd":"joeword","userId":"Joe"},"Sue":{"userPwd":"sueWord","userId":"Sue"},"name":"Administration","Tim":{"userPwd":"timWord","userId":"Tim"}}
Done exporting group in json to admin.json
Importing group from admin.json
names are: Joe, Sue, name, Tim,
Group: Administration has users: Joe, Sue, Tim,
Used Java serialization of the group to admin.ser
Done importing the group from admin.ser as:
Group: Administration has users: Tim, Joe, Sue,

Deprecated Gradle features were used in this build, making it incompatible with Gradle 8.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

See https://docs.gradle.org/7.6/userguide/command_line_interface.html#sec:command_line_warnings

BUILD SUCCESSFUL in 1s
2 actionable tasks: 2 executed
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Serialization\GroupJson> |

```

Description:

This is an example of object serialization. It manages a group of users and saves data to JSON and ser files. The console prints the program's progress as it performs each task as it exports the group's state and reconstructs Java objects by reading them back into memory.

2.4 - Second System

EC2 Server:

```
[ec2-user@ip-172-31-42-253 JavaSimpleSock2]$ java -cp build/classes/java/main SockServer
Server ready for 3 connections
Server waiting for a connection
Received the String HI
Received the Integer 100
Server waiting for a connection
```

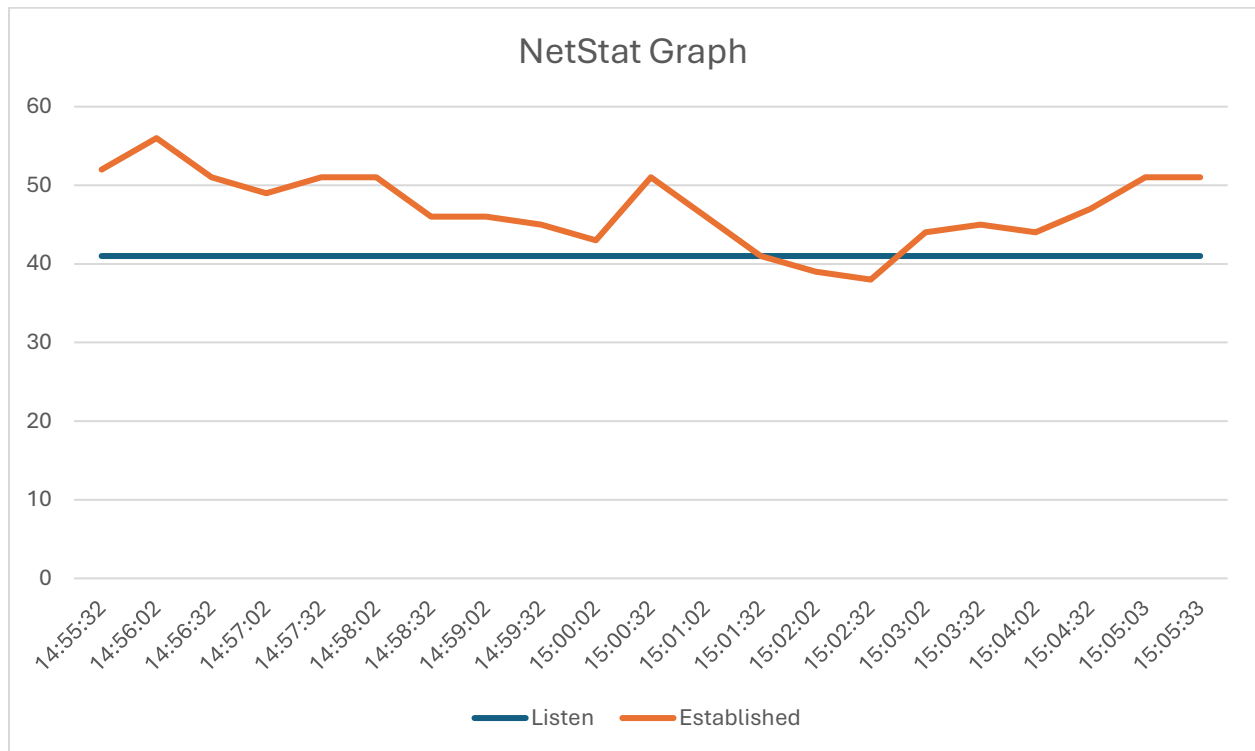
Client (Windows):

```
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Sockets\JavaSimpleSock2> java -cp build\classes\java\main SockClient 3.134.92.0
Got it!
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Sockets\JavaSimpleSock2> |
```

Screencast link: https://arizonastateu-my.sharepoint.com/:v:/g/personal/tjhanse9_sundevils_asu_edu/IQBY8oABp-cUTqD51_nYOaaKAURFM0zWoWsMYoSRmdQq7xl?e=u4eYmh

Part 2: Networking

3.1 - TCP Network Sockets



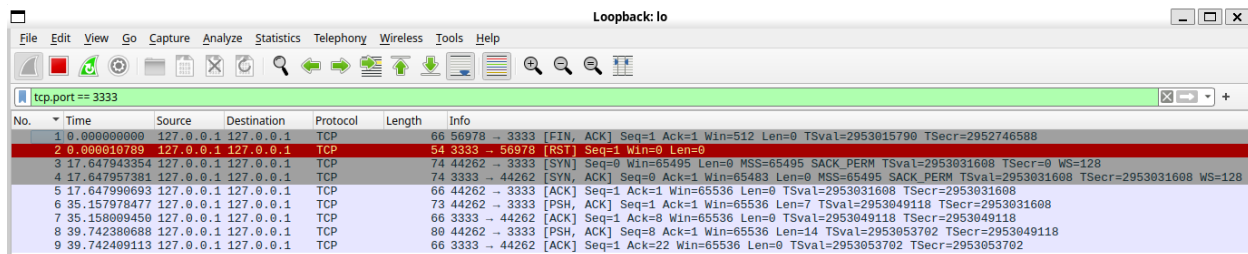
Script used (Windows PowerShell):

```
while ($true) {  
    $time = Get-Date -Format "HH:mm:ss"  
    $listen = (netstat -an | Select-String "LISTENING").Count  
    $established = (netstat -an | Select-String "ESTABLISHED").Count  
    "$time, $listen, $established" | Out-File -Append -FilePath "socket_data.csv"  
    Write-Host "Logged at $time - Listen: $listen | Established: $established"  
    Start-Sleep -Seconds 30  
}
```

3.2 - Sniffing TCP/UDP Traffic

Captured in WSL Environment

TCP Wireshark Results:



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	66	56976 → 3333 [FIN, ACK] Seq=1 Ack=1 Win=512 Len=0 TSval=2953015790 TSecr=2952746588
2	0.000010789	127.0.0.1	127.0.0.1	TCP	54	3333 → 56976 [RST] Seq=1 Win=0 Len=0
3	17.647943354	127.0.0.1	127.0.0.1	TCP	74	44262 → 3333 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=2953031608 TSecr=0 WS=128
4	17.647957381	127.0.0.1	127.0.0.1	TCP	74	3333 → 44262 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=2953031608 TSecr=2953031608 WS=128
5	17.647990693	127.0.0.1	127.0.0.1	TCP	66	44262 → 3333 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=2953031608 TSecr=2953031608
6	35.157978477	127.0.0.1	127.0.0.1	TCP	73	44262 → 3333 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=7 TSval=2953049118 TSecr=2953031608
7	35.158009458	127.0.0.1	127.0.0.1	TCP	66	3333 → 44262 [ACK] Seq=1 Ack=8 Win=65536 Len=0 TSval=2953049118 TSecr=2953049118
8	39.742380608	127.0.0.1	127.0.0.1	TCP	80	44262 → 3333 [PSH, ACK] Seq=0 Ack=1 Win=65536 Len=14 TSval=2953053702 TSecr=2953049118
9	39.742409113	127.0.0.1	127.0.0.1	TCP	66	3333 → 44262 [ACK] Seq=1 Ack=22 Win=65536 Len=0 TSval=2953053702 TSecr=2953053702

a) Explain both the commands you used in detail. What did they actually do?

- “nc -k -l 3333” -l sets Netcat to listen on port 3333. The -k keep-alive flag tells the server to stay open and continue listening.
- “nc 127.0.0.1 3333” initiates a Netcat connection to 127.0.0.1 on the port 3333 the other terminal is listening on.

b) How many packets were sent back and forth so the client/server could send/receive these two lines of data?

4 packets, numbers 6, 7, 8, and 9.

c) How many packets were needed back and forth to capture the whole "process"?

9 packets total.

d) How many bytes is the data (only the data) that was sent from client to server?

21 bytes total. 7 for “SER32”, 14 for “is amazing!!!”

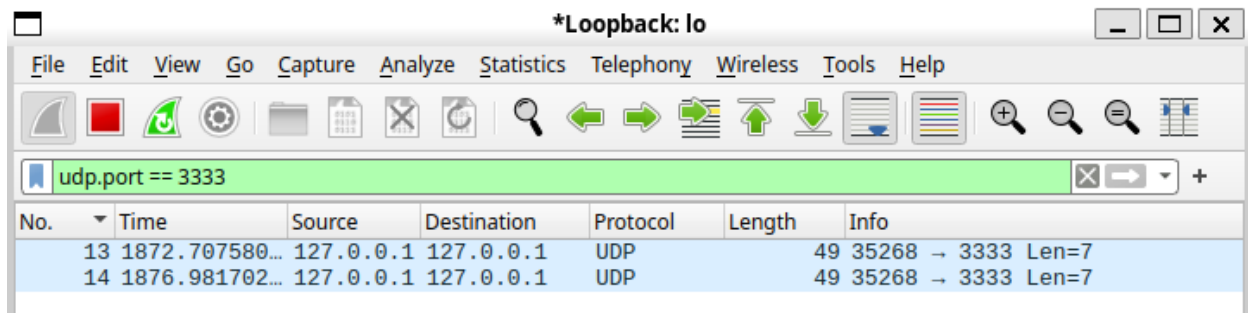
e) How many total bytes went over the wire (back and forth) for the whole process?

619 bytes.

f) How much overhead was there?

598 bytes.

UDP Wireshark Results:



The image shows a Wireshark window titled '*Loopback: lo'. The filter bar at the top shows 'udp.port == 3333'. The packet list shows two packets, both of length 49 bytes. The packet details pane shows the structure of a UDP packet: Ethernet II (14 bytes), Internet Protocol Version 4 (20 bytes), and User Datagram Protocol (8 bytes). The data field shows 7 bytes of data.

No.	Time	Source	Destination	Protocol	Length	Info
13	1872.707580...	127.0.0.1	127.0.0.1	UDP	49	35268 → 3333 Len=7
14	1876.981702...	127.0.0.1	127.0.0.1	UDP	49	35268 → 3333 Len=7

a) Explain both the commands you used in detail. What did they actually do?

- “nc -u -l 3333” same as TCP command, but -u flag tells Netcat to use UDP instead of default TCP.
- “nc -u 127.0.0.1 3333” initiates a UDP Netcat connection to 127.0.0.1 on the port 3333 the other terminal is listening on.

b) How many packets were needed to capture those 2 lines?

2 packets.

c) How many packets were needed to capture the whole "process"?

2 packets total.

d) How many bytes is the data (only the data) that was sent?

14 bytes total.

e) How many total bytes went over the wire?

98 bytes.

f) How many bytes was the whole process compared to the actual data?

98 bytes total minus 14 bytes of actual data equals 84 bytes overhead.

g) What is the difference in relative overhead between UDP and TCP and why?

TCP required 619 bytes compared to just 98 bytes sent over UDP. TCP handshake took extra packets to handshake before sending data and close the connection after. This is skipped in UDP. The TCP trace keeps track of more information- flags, sequence numbers and window sizes. UDP only shows the connection ports and the data length. It doesn't track if the data actually arrives or its arrival order.

3.3 - Running client servers in different ways

Task 3.3.1

Local Server

```
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Sockets\JavaSimpleSock2> java -cp
build\classes\java\main SocketServer
Server ready for 3 connections
Server waiting for a connection
Received the String HI
Received the Integer 100
Server waiting for a connection
```

Local Client

```
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Sockets\JavaSimpleSock2> java -cp build\classes\java\main SockClient localhost
Got it!
PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Sockets\JavaSimpleSock2>
```

Screencast link: https://arizonastateu-my.sharepoint.com/:v/g/personal/tjhanse9_sundevils_asu_edu/IQAgDpswtVinR4RWVfbH4X6nAc7OvgKujwGLj_UrLI7lshA?e=bUvpeT

Task 3.3.2

AWS Server

```
#  
~\##### Amazon Linux 2023  
~~\#####\  
~~\###|  
~~\#/  
~~V~' '~>  
~~~~  
~~.-.  
~/m/' ~
```

Last login: Tue Mar 24 19:34:02 2026 from 3.16.146.3
[ec2-user@ip-172-31-42-253 ~]\$ cd ser32lexamples/Sockets/JavaSimpleSock2
[ec2-user@ip-172-31-42-253 JavaSimpleSock2]\$ java -cp build/classes/java/main SockServer
Server ready for 3 connections
Server waiting for a connection
Received the String HI
Received the Integer 100
Server waiting for a connection

Local Client



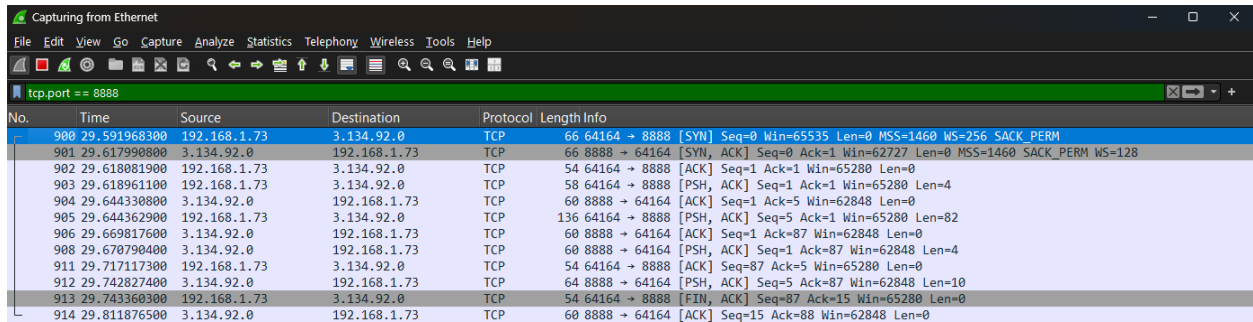
The screenshot shows a Windows PowerShell terminal window. The title bar reads "Windows PowerShell". The output text is as follows:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\iamth\OneDrive - Arizona State University\SER 321\Git Repos\ser321examples\Sockets\JavaSimpleSock2> java -cp
build\classes\java\main SocketClient 3.134.92.0
Got it!
```

Wireshark from Local to AWS connection



The image shows a Wireshark packet capture window titled "Capturing from Ethernet". The filter bar at the top shows "tcp.port == 8888". The packet list table below shows 14 packets. The first packet (No. 900) is a SYN packet from 192.168.1.73 to 3.134.92.0 on port 8888. The subsequent packets (901-914) show the connection establishment process, including ACKs, PSHs, and data transmission. The final packet (914) is an ACK packet from 3.134.92.0 to 192.168.1.73.

No.	Time	Source	Destination	Protocol	Length	Info
900	29.591968300	192.168.1.73	3.134.92.0	TCP	66	64164 → 8888 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
901	29.617990800	3.134.92.0	192.168.1.73	TCP	66	8888 → 64164 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM WS=128
902	29.618081900	192.168.1.73	3.134.92.0	TCP	54	64164 → 8888 [ACK] Seq=1 Ack=1 Win=65280 Len=0
903	29.618961100	192.168.1.73	3.134.92.0	TCP	58	64164 → 8888 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=4
904	29.644330800	3.134.92.0	192.168.1.73	TCP	60	8888 → 64164 [ACK] Seq=1 Ack=5 Win=62848 Len=0
905	29.644362900	192.168.1.73	3.134.92.0	TCP	136	64164 → 8888 [PSH, ACK] Seq=5 Ack=1 Win=65280 Len=82
906	29.669817600	3.134.92.0	192.168.1.73	TCP	60	8888 → 64164 [ACK] Seq=1 Ack=87 Win=62848 Len=0
908	29.670790400	3.134.92.0	192.168.1.73	TCP	60	8888 → 64164 [PSH, ACK] Seq=1 Ack=87 Win=62848 Len=4
911	29.717117300	192.168.1.73	3.134.92.0	TCP	54	64164 → 8888 [ACK] Seq=87 Ack=5 Win=65280 Len=0
912	29.742827400	3.134.92.0	192.168.1.73	TCP	64	8888 → 64164 [PSH, ACK] Seq=5 Ack=87 Win=62848 Len=10
913	29.743360300	192.168.1.73	3.134.92.0	TCP	54	64164 → 8888 [FIN, ACK] Seq=87 Ack=15 Win=65280 Len=0
914	29.811876500	3.134.92.0	192.168.1.73	TCP	60	8888 → 64164 [ACK] Seq=15 Ack=88 Win=62848 Len=0

To set up this connection, I followed the same process from the previous part, but instead of connecting via localhost, I ran SockClient from the AWS IP address. On Wireshark, I switched from monitoring Loopback for local communication to Ethernet.

3.3.2 - 3.3.3 Client on AWS

In order to run the server locally through my public IP address and the client through the AWS server, I would need to port-forward port 8888 on my modem and allow the AWS server to pass through my firewall to communicate, these act as extra security to protect my home internet. After making these changes, my router would direct traffic coming through my public IP through this port to my local machine and the connection would work the same.